

Machine Learning Week 4

By 21337470

Introduction

In this report, I will be detailing my implementation of the Week 4 Machine Learning Module assignment. We have used Logistic Regression and kNN models alongside evaluation metrics like Confusion Matrices, Accuracy Scores, ROC Curves (Receiver Operating Characteristic), Baseline classifiers such as most frequent class and random guessing methods.

This work has been done in a Jupyter Notebook Environment and given the code accordingly.

The dataset used is : # id:23-23-23-1

This is my understanding of these methods:

Logistic Regression

Logistic Regression is a linear model used for binary classification problems (0 or 1, true or false outputs). It works by estimating the probability that a given input belongs to a certain class (usually class 1). It uses the logistic function (or the sigmoid function) to squeeze the output between 0 and 1. The model outputs the probability of a sample belonging to the positive class, and based on a threshold (usually 0.5), we can classify the sample as either class 0 or class 1.

k-Nearest Neighbors (kNN)

kNN is a non-parametric classification algorithm. It works by finding the 'k' closest samples to a given input and assigning the class based on a majority vote from its neighbours. The model does not have a training phase per se but uses the distance between samples in the dataset to make predictions at runtime. In my implementation, I chose $k=17$ for the dataset 1 and $k = 6$ for the second dataset, which gave the best accuracy based on cross-validation.

Confusion Matrices

These matrices provide a breakdown of true positives, true negatives, false positives, and false negatives, which are helpful in understanding the model's strengths and weaknesses.

True Positive: When the model prediction and the actual class of the sample is the same as 1 (positive/1 value). A correct prediction so to say.

True Negative: When the model prediction and the actual class of the sample is the same as 0 (negative/0 value). A correct prediction.

False Positive: When the model prediction is 1(positive/1 value) and the actual sample value as 0 (negative/0 value). An incorrect prediction, where the model predicts a positive and the actual value is negative.

False Negative: When the model prediction is 0(negative/0 value) and the actual sample value as 1 (positive/1 value). An incorrect prediction, where the model predicts a negative and the actual value is positive.

Accuracy Score

A basic metric that tells us the proportion of correct predictions out of the total predictions made.

ROC Curves

These plot the true positive rate against the false positive rate at different thresholds, giving a good visual representation of model performance. The area under the curve quantifies this performance, with higher values indicating better performance.

Most Frequent Classifier

This classifier simply predicts the most frequent class from the training data.

Random Classifier

This classifier makes random predictions. Like the most frequent classifier, it acts as a reference to ensure that the trained models are doing better than random guessing.

Setup

[Code Index: 1](#)

At first, I have imported all the necessary libraries to be used in this project. I imported the numpy library for dataset manipulation and computations. Then I have imported pandas for loading data into dataframes for loading into the model and I imported Matplotlib library for visualizing the graphs.

I have also imported many functions from the Sklearn for the assignment,

From the preprocessing class of Sklearn, I imported PolynomialFeatures which is used to convert simple terms into complex polynomials to capture more complex data in the models. From the models class of Sklearn, I imported the logistic regression model which I explained above. From the neighbours class of Sklearn, I imported the KNeighboursClassifier, the kNN model used in the report.

Also from Sklearn library, from model_selection class, I imported kfold and test_train_split. Kfold is used for the cross validation for splitting the dataset into k number of sets or folds.

Test_train_split function is used to split the dataset into the training and tests set for the model to use.

Now from the sklearn library, from the metrics class, I imported accuracy_score, confusion_matrix, ConfusionMatrixDisplay, roc_curve and auc. Accuracy_score gets the accuracy score of the model by simply dividing the correct predictions of the model by the actual values. Confusion_Matrix and ConfusionMatrixDisplay are both used to display the confusion matrix as explained in the earlier section. The roc_curve calculates the false positive and true positive rates for different thresholds, helping us plot the ROC curve and the AUC, (Area Under Curve) is used to summarize the performance of a model using the ROC curve. The higher the AUC, the better the model is at distinguishing between the classes.

I also imported the DummyClassifier from the dummy class in sklearn library, which acts as a baseline model for the report. It makes simple guesses on the dataset, using random guesses or frequent class, To be used for comparison against our models.

Finally I imported warnings in order to suppress some depreciation warnings in the output.

Part (i). A

Code Index: 2

First, we load the dataset using `pd.read_csv()` and assign column names: `X_1`, `X_2`, and `Target`. These represent the two features (`X_1` and `X_2`) and the target variable (`Target`). The features are extracted into `X`, while the target is stored in `y`.

Then I used KFold cross-validation with 5 splits (i.e., the data will be divided into 5 parts, with each part taking a turn as the test set). I also added `shuffle=True` to randomize the data before splitting it into folds. The `random_state=42` ensures that I get the same splits each time I run the code as it is stored as a seed.

then I specified the degree array upto 5 to create the polynomial features upto the 5th degree. Then I initialized the `C_values` and `np.logspace(-3, 3, 7)` generates 7 values between 10^{-3} and 10^3 , to explore `C` values in this range.

Then I initialized results dictionary to store the results.

Then, using a for loop in the in the degree array to create polynomial values to create `PolynomialFeatures`.

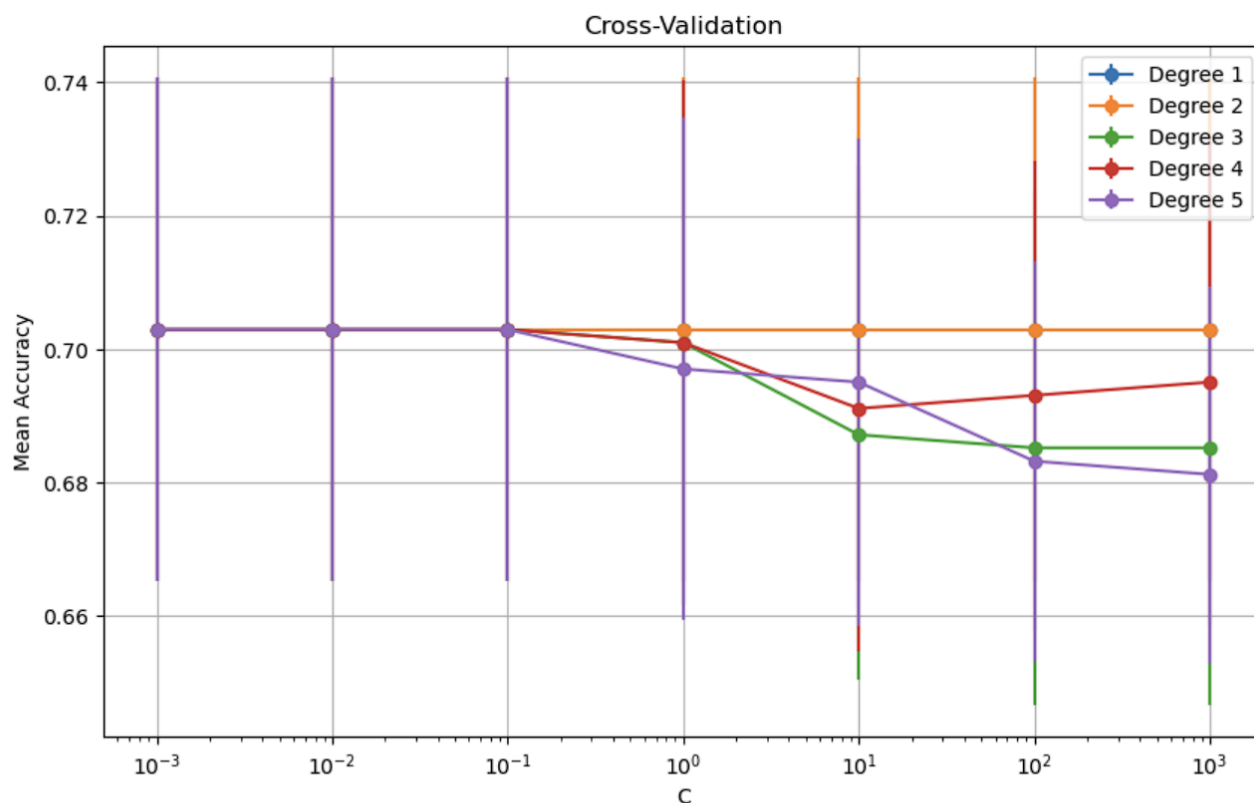
Then in another nested for loop over `C_values`, I first initialized `accuracy_scores` array to store the accuracy values. Now in another for loop for every train and test index in the `kf.split`, I trained the logistic regression model over 10000 iterations for each split of the data in order to perform cross validation over the dataset and then I used the accuracy score function to get the accuracy score of each split of the function.

And then in the next lines I have stored these results in the aforementioned results dictionary. Then for displaying the results, I stored the results in the `results_df` dataframe of the results.

In the next lines, I have used yet another for loop to plot the errorbar in the plot from the results dataframe and used the `plt` functions to display the labels, legend and used `.show` to visualize the plot.

Finally, I found the best combination of polynomial degree and regularization strength by looking for the highest mean accuracy in `results_df`. I printed out the best degree and `C` value, which will be used to train the final model.

The Output



Best Polynomial Degree: 1
Best C: 0.001

Now from this graph, we can see the performance of different degrees of polynomial for different values of C. The output tells us that the best hyperparameters (externally defined parameters) are Polynomial Degree 1 for $C = 10^{-3}$. However, for higher values of C, we can see that most polynomial degrees do not perform well and tend to overfit. Across the model, degree 1 and degree 2 both perform consistently well across different values of C, however as the output suggest, the best hyperparameters at a single point are $C = 10^{-3}$ and degree 1. However, the degree 2 polynomials are also a good alternative with similarly well and stable results.

Part i. (b)

[Code Index: 3](#)

In this code, first I made a range of K values from 1 to 21 to test the kNN classifier over a range of values to find the best K from the range of all possible k values, and then I setup a dictionary like before to store the result values. Then I used the warnings call to suppress the the depreciation warnings on running.

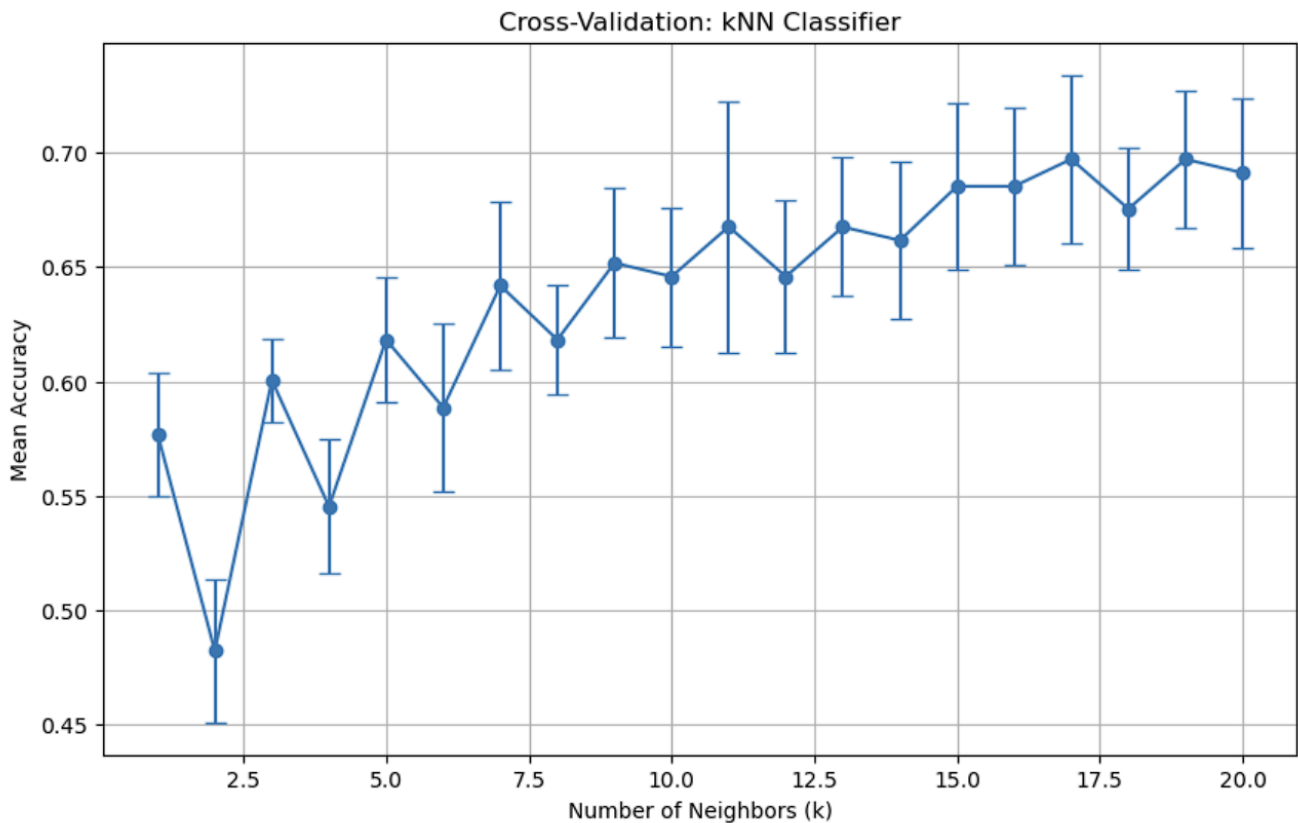
Then I iterated over the K values in a for loop, in a nested for loop I split the training and test values, then using the knn variable, I called the KNeighboursClassifier and specified the number of neighbours to be considered as the current K value, then used the .fit to pass the dataset through the model. Then used y_pred_cv to get the prediction output to pass through the accuracy_score variable and get the accuracy score of the current model in loop by passing the predicted and actual variables to get the score of the proportion, then just like before, I stored the results in the result dictionary and then put them into a dataframe for visualizing the resultant values. Then I visualized the graph using the plt function to plot the grid of the K number of neighbours with the resulting accuracy score.

Now to find the best k value, using np.argmax() We use that to find the highest argument value of the result argument index, to get the best accuracy score value of mean value of k in the next lines with its corresponding accuracy score.

[Code Index: 4](#)

Finally at the end, I called the model and .fit again to train and predict the final resultant model with the best value of k and printed the model's accuracy score.

The Output



```
Best k: 17
Highest Cross-validation Accuracy for kNN: 0.6970
KNeighborsClassifier(n_neighbors=17)
```

Now as seen in this graph, for lower values of K nearest neighbours, the model and its subsequent accuracy score are quite unstable and fluctuating, as the values of k get higher, the model seems to stabilize in accuracy scores, as the model outputs and as seen in graph, the value of K nearest neighbours with the highest accuracy score value is k=17 with an accuracy of 0.6970 or about 70%, which our program successfully outputs.

Test Accuracy for kNN (k=17): 0.6863

As we can see the accuracy score for the best k to be printed, which is k=17.

Part i. (C)

[Code Index: 5](#)

At first using the `test_train_split` function and then 0.2 test size to split the training and test data into 80% training and 20% testing split.

Then specifying the logistic regression's with the best hyperparameters I found earlier in part (a), those being the best degree 1 and C value = 0.001 to train the linear logistic regression, then I passed the dataset through the model and got its predictions.

Now the next part is to calculate the confusion matrix for its confusion matrix plots, to do that I passed the `y_test` and the predicted `y_log_reg_pred` values and then using `ConfusionMatrixDisplay` from the `matplotlib` library to initialize the confusion matrix.

Now next I set up the kNN model using `k=17` (which as determined earlier, was the best value of K of kNN for dataset1), and then `.fit` through the dataset in the model and used the `.predict` the value of the kNN model.

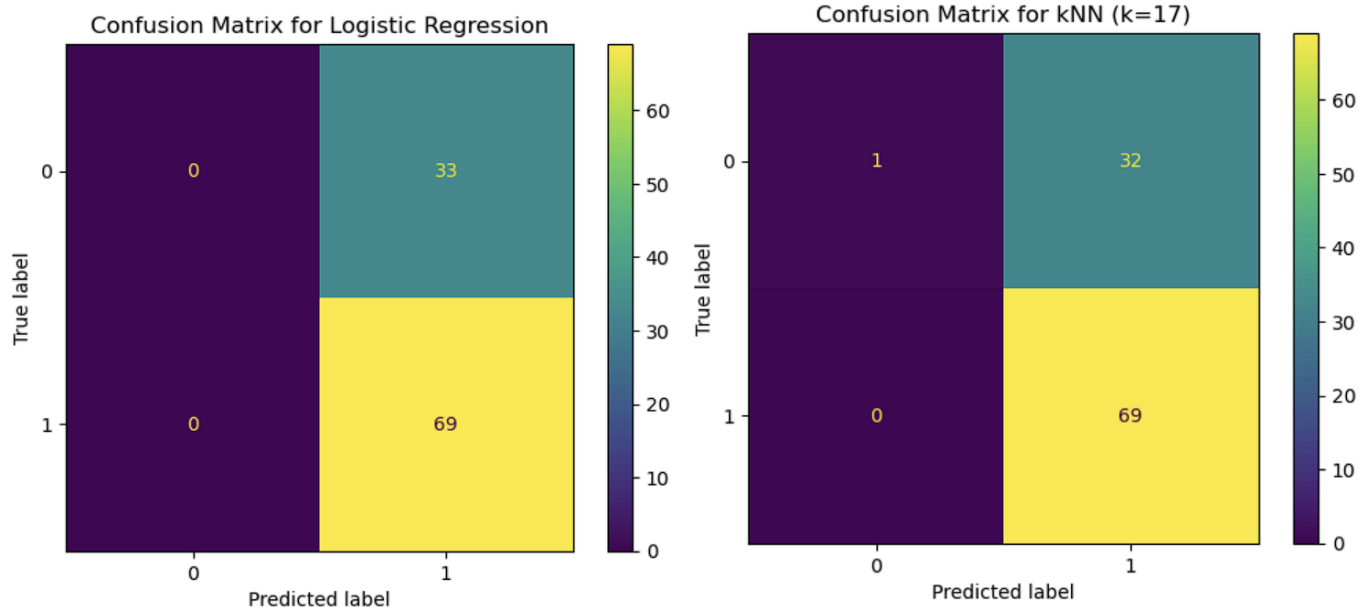
Now finally, to display the confusion matrix for the kNN model, similarly to the logistic regression confusion matrix, I passed through the `y_test` and `y_knn_pred`, and similarly the `confusionmatrix` function and `.plot` to display the resulting confusion matrix.

Code Index: 6

Now I coded in the dummy baseline models, both most frequent class and random guess models for comparing the output. To do this, I used the `DummyClassifier` function to specify the model by passing in the most frequent model strategy into the dummy model function and `.fit` to pass in the dataset and `.predict` to get the output of this `dummyclassifier` model. To display the confusion matrix for this model, using the same code in confusion matrix, I put in the test values and the predictions from this model and then displayed the confusionmatrix plot.

Now next, I did the same thing for the random guess model, but instead of putting in most frequent as the strategy parameter, I used `uniform`, which specifies a uniform distribution in the random guess model. Using the same code as used before, I displayed the confusion matrix for the same.

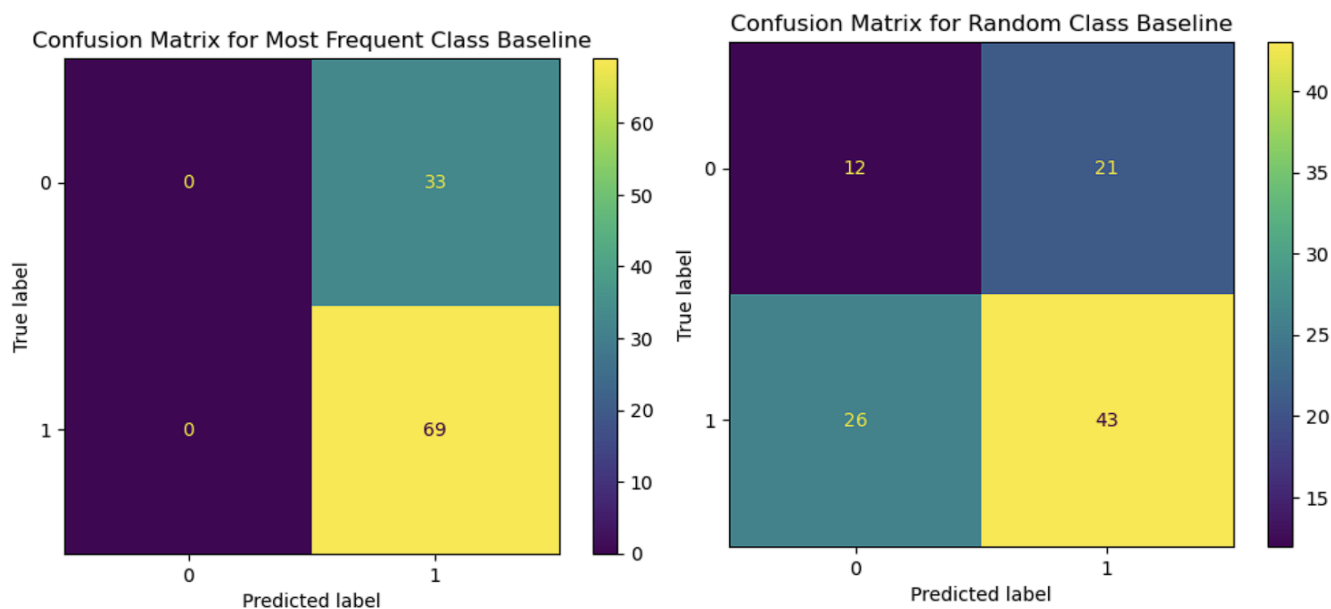
The Output



Now seeing the confusion matrices, we see that both models have correctly predicted 69 True Positives, then the logistic regression has 33 False Positives and kNN has predicted 32 false positives, so both models have relatively high number of false positives.

Now True Negatives, the kNN model has 1 true negative and logistic regression has 0 and then both models have 0 False Negatives.

All in all, both models, by their confusion matrices, give predictions quite similarly with very few differences in between.



Now here we see the confusion matrices for most frequent class and random class baseline models, As we can see that the most frequent class performs the same as the logistic regression model, although the random guess model performs poorly in comparison to the rest of the models.

Part (i). D

[Code Index = 7](#)

Now to get the ROC Curves of both models, first, repeating the code from the previous parts, I specified the logistic regression model, kNN model with best $k=17$ and the two earlier baseline models from the last part are re-specified with the same code, the most frequent class and random guess models.

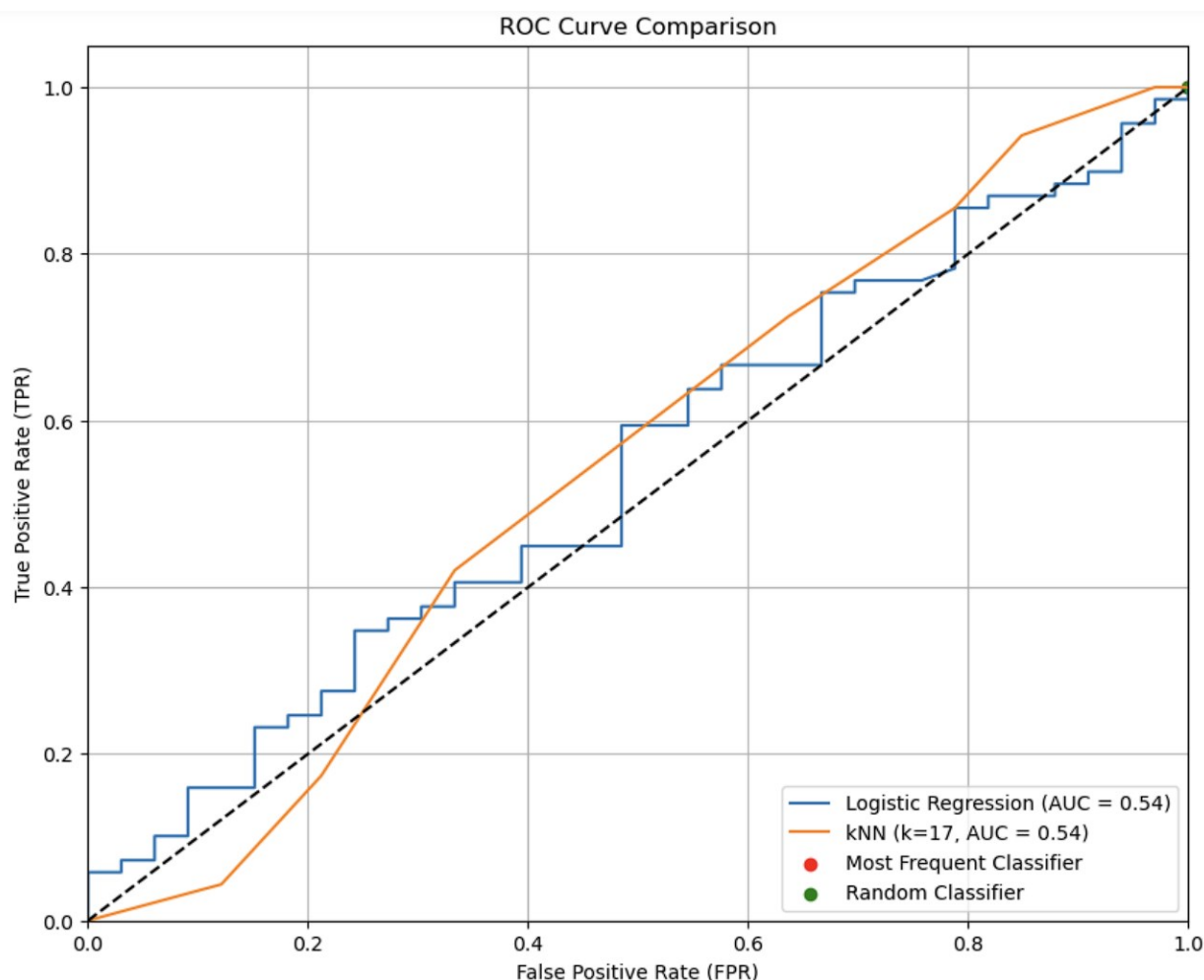
Now to plot the ROC Curve function, which calculates the True Positive Rate and False Positive Rates for various threshold values and get the Area Under Curve.

So at first I Instantiated the variables for TPR and FPR to be instantiated with the `roc_curve` function with `y_test` and `y_log_reg_probs` passed in, now to get the Area Under Curve from the `auc` function with the TPR and FPR values passed in. Then I plotted the plot by using `plt.plot()` with the FPR and TPR values for Logistic Regression, and added a label showing the AUC value as well. I did the same for the kNN model, calculating the TPR and FPR for the predictions made by the kNN classifier, and again passing these into the `auc()` function to get the AUC, and plotting it similarly.

Next, for the Most Frequent Classifier and Random Classifier, since they don't generate full ROC curves, I plotted their results as scatter points. I extracted their FPR and TPR values from the first element of the `roc_curve()` output and plotted them as distinct red and green points on the graph for easy visualization.

Lastly, I added a diagonal dashed line from `(0, 0)` to `(1, 1)` to represent random guessing, a standard addition to ROC plots. I adjusted the x and y axis limits to make sure all points are visible, added gridlines for clarity, and a legend in the lower right corner to distinguish between all models.

The Output



Both the Logistic Regression and the kNN model ($k=17$, the best k Value) have an AUC of 0.54, which is quite low, just marginally better than a random guess (AUC = 0.50). This is reflected in their ROC curves, which are very close to the diagonal line, meaning that neither model is particularly effective at distinguishing between the classes for this dataset.

The Most Frequent Classifier and Random Classifier are plotted as single points on the curve. The Most Frequent Classifier shows up with minimal TPR (as it likely predicts the majority class only), and the Random Classifier sits almost exactly on the diagonal line, as expected.

Overall, the ROC plot indicates that neither the Logistic Regression nor kNN model perform well on this dataset. Both models are struggling to make meaningful predictions beyond random chance, which is reflected by their AUC scores.

Part (i). E

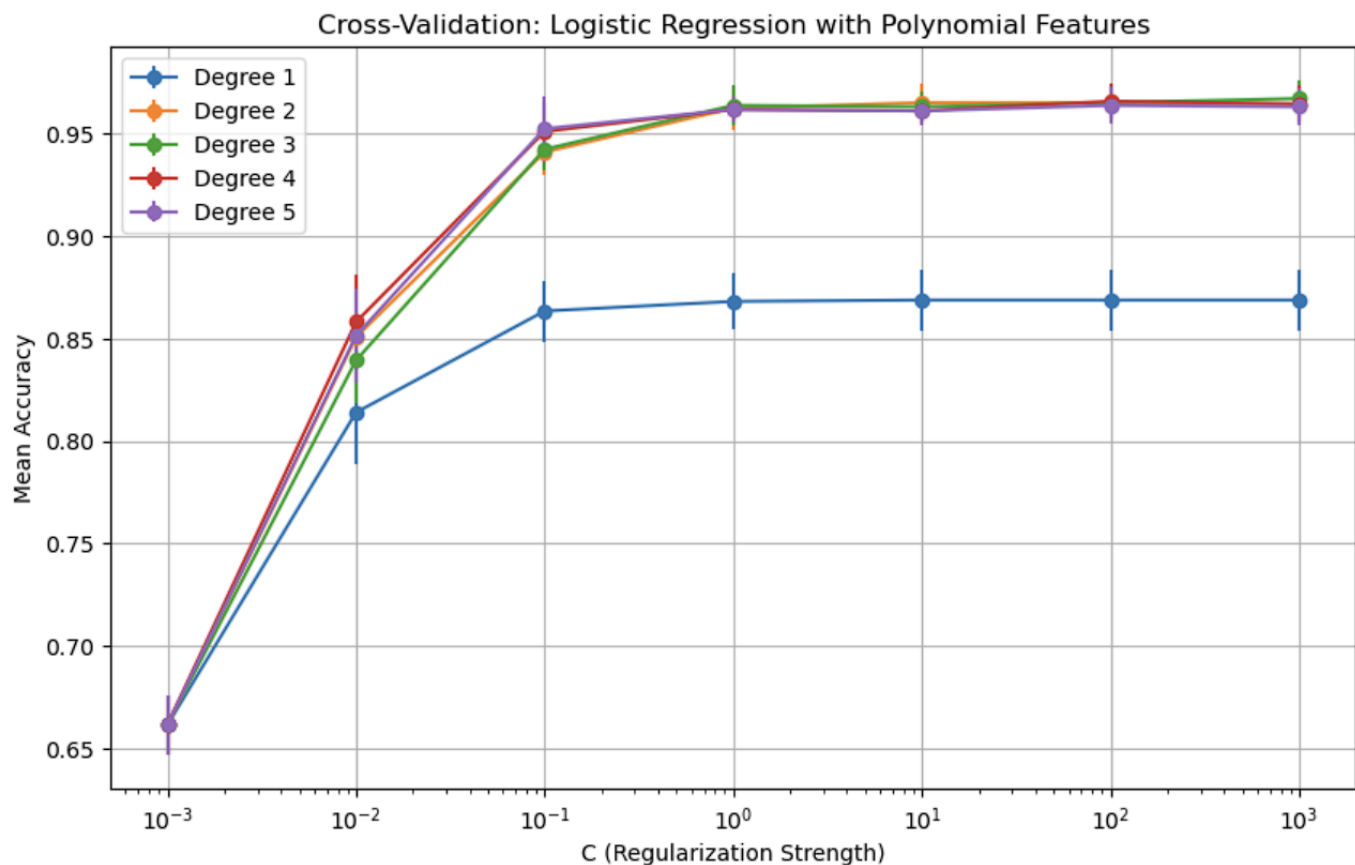
Now looking out the output from part C and D, we can see that our logistic regression and kNN models, despite taking their best hyperparameter values, do not perform much better than the baseline most frequent class model. From the confusion matrix, we can see that the confusion matrix of logistic regression and most frequent class baseline model scores are the same.

Even from the ROC curve graph, we can see, using AUC values that the models have value of 0.54, which is not better than the baseline model's auc being 0.5. Hence, concluding these findings from both C and D part outputs, I think the best model for this particular dataset would be the Most Frequent Class Baseline model, since the accuracy of both baseline and our models is almost the same and the trained models only give an accuracy increase of about 0.04 AUC value at most. So, using a model on this dataset with such less accuracy increase would not be worth it and it might be better to use a simple baseline classifier like Most Frequent Class.

Part (ii)

Now, For part 2, I used the second dataset and repeated the steps of Part (i), since the code remains exactly the same except the dataset file is dataset2.csv and the best values of the models will be different, I will only explain the output of the code.

Part (ii). A

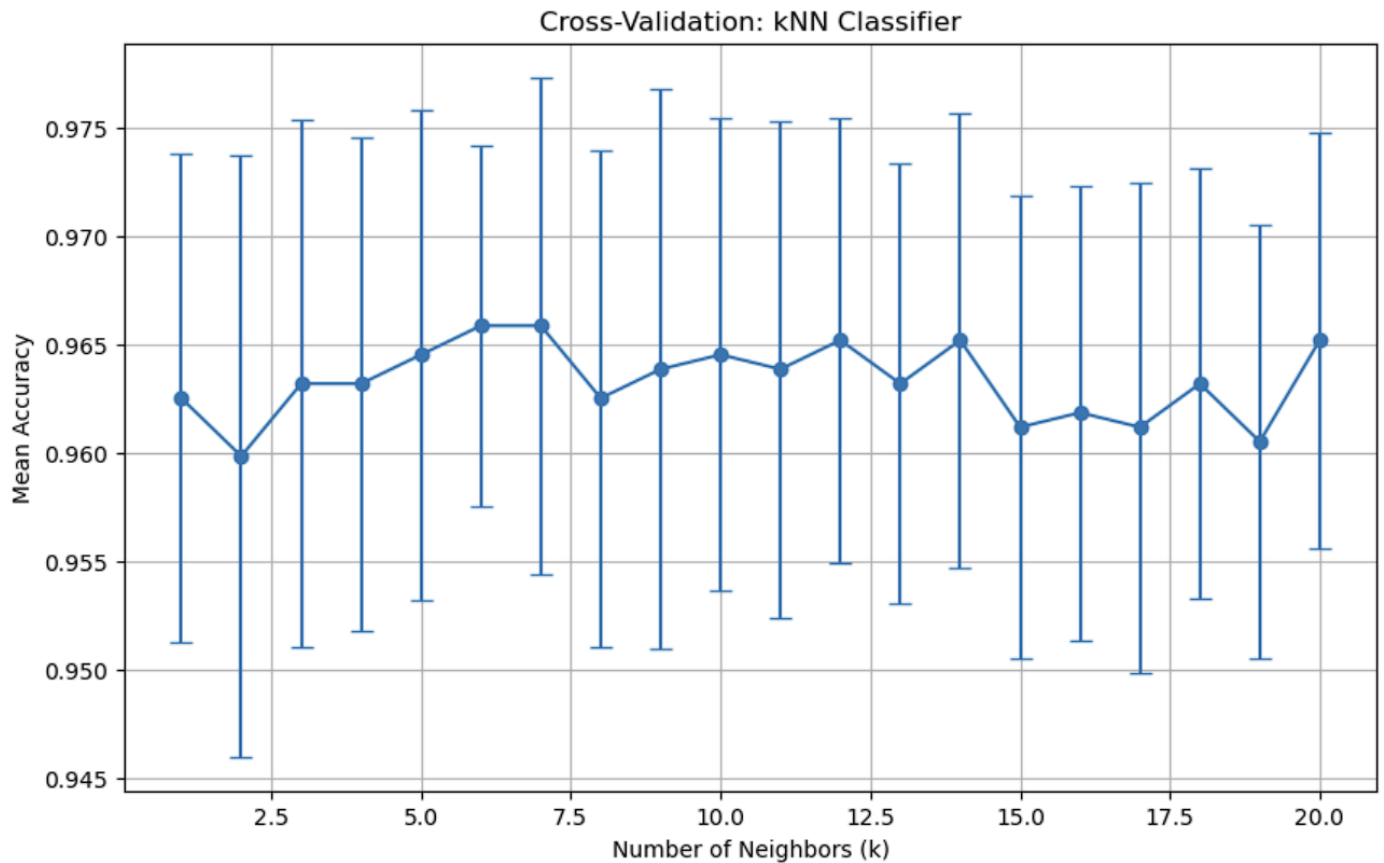


Best Polynomial Degree: 3
 Best Regularization Strength (C): 1000.0

In this graph for dataset2, we again see the cross validation results for different polynomial degrees and the mean accuracy offered by them plotted on the graph. Now from the output, we can see the best polynomial degree 3 and best value of C (regularization strength) being 1000. Although 3 performs well, degree 4 and 5 perform just as well as degree 3.

The degree 1 is does not perform well while higher level polynomials show a better increase in the accuracy. The model does not perform well for lower values of C, the accuracy for all degrees increase from $C=10^{-3}$ to $C=0$ and stabilize at $C=0$ after which all degrees of the polynomial values seem to stabilize at $C=0$.

Part ii. B



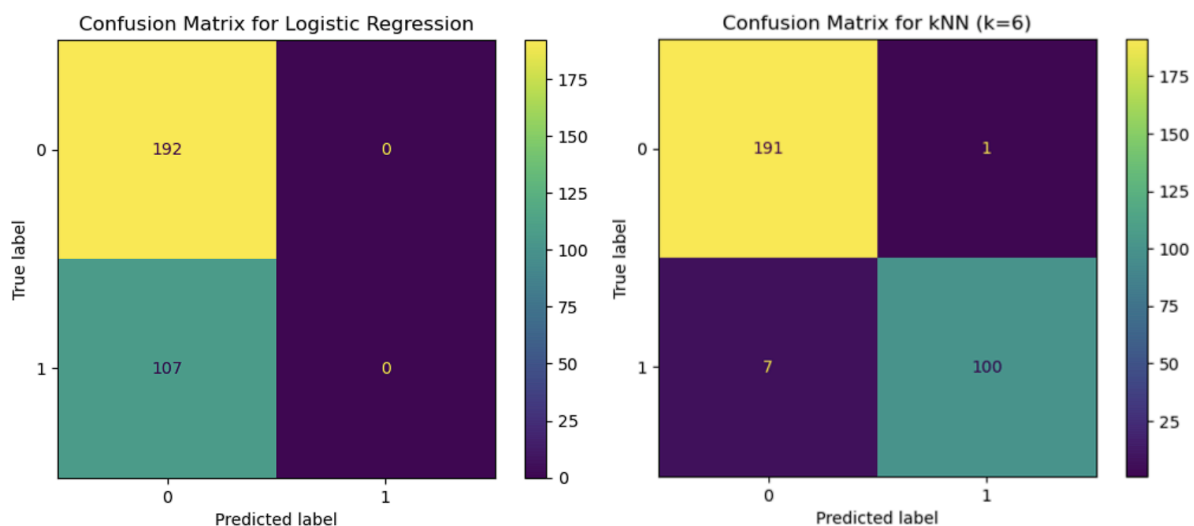
Best k: 6
 Highest Cross-validation Accuracy for kNN: 0.9659
`KNeighborsClassifier(n_neighbors=6)`

Now in this graph, We see the dataset 2 for values of K. As seen in the output the values of K do not change the accuracy of the model by much. As seen in the output, the best value of K is 6 with the accuracy coming out to be 0.9659 or 96%. However the accuracy does not vary much by the value of K. In the next upcoming parts, the value of $k = 6$ will be used for this dataset for the kNN model.

Test Accuracy for kNN ($k=6$): 0.9732

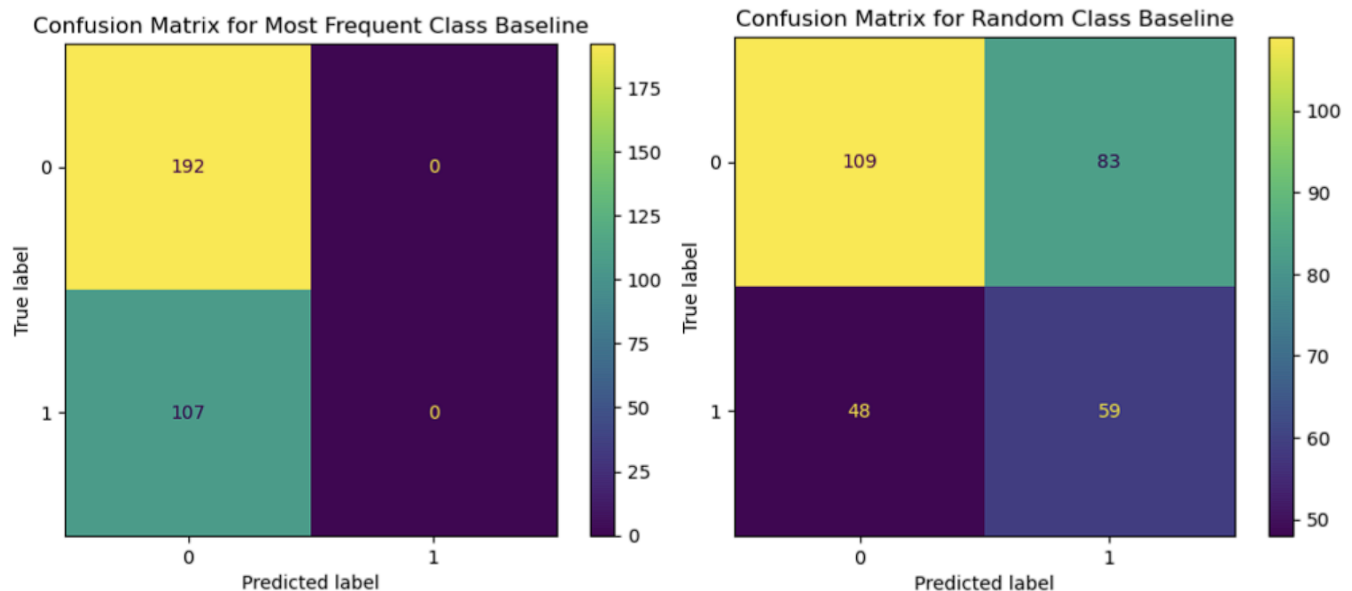
As we can see, the model performs quite well, with $k=6$, giving an accuracy of 0.9732 or 97%.

Part ii. C



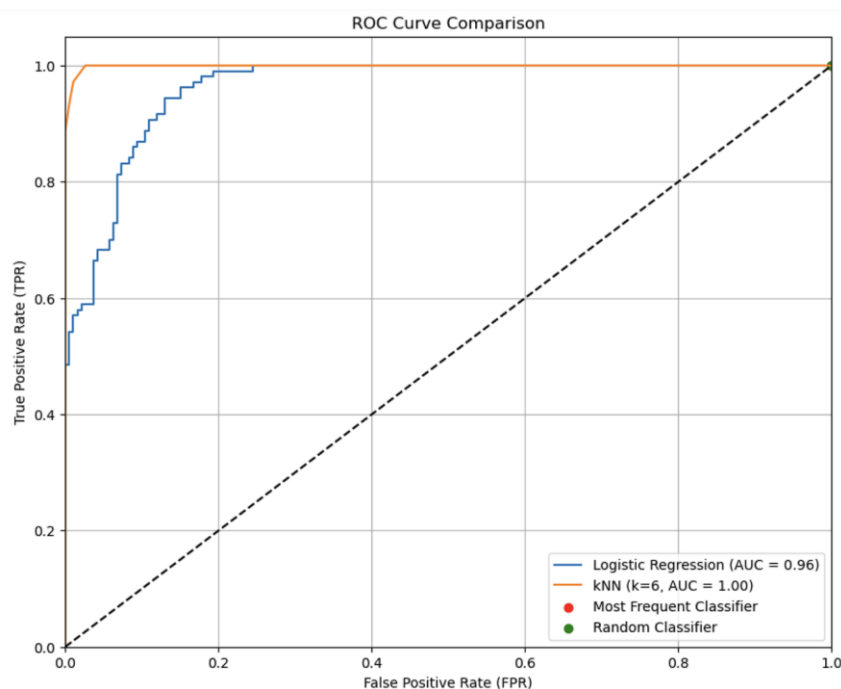
Now in these confusion matrices, we can see that Logistic Regression model does not perform well on dataset 2, it has heavily misclassified the class 1 instances and has 107 False Negatives with 192 True Negatives. It has not classified any True Positives or False Positives.

Thankfully, for the kNN model we get much better accuracy of the model. With only 8 misclassifications (1 False Positive and 7 False Negatives) and 100 True Positives and 191 True Negatives. Thus we can see that k=6 is a good choice for the kNN model.



Now we observe the accuracy of the baseline models, As seen the Most Frequent Class Baseline performs as badly as the Logistic Regression Model, with 192 True Negatives and 107 False Negatives (with 0 False Negatives and 0 True Positives). Now observing the random class baseline, as before it performs poorly on the dataset with the random guesses with 109 True Negatives, 83 False Positives, 48 False Negatives and 59 True Positives.

Part ii. D



Now looking at ROC Curve graph, we see that the Logistic Regression model has an AUC = 0.96, which is pretty well for the model itself, although surprisingly the kNN model has a perfect score of AUC 1.00. Unsurprisingly, the baseline classes have AUC of 0.5.

In summary, both logistic regression and kNN models are far superior to these baseline models, with kNN being the standout performer with a perfect classification. The baseline classifiers, as expected, offer no real predictive power.

Part (ii). E

Now looking at the output for Dataset 2, we observe quite different results compared to Dataset 1. Starting with the confusion matrices, our kNN model ($k=6$) and Logistic Regression model show much better performance than the baseline models. Specifically, kNN achieved a near-perfect classification with very few misclassifications, reflected in the confusion matrix where there are minimal false positives and false negatives. Similarly, logistic regression also performs well, though it doesn't quite reach the level of accuracy as kNN, but it still far surpasses the baseline models.

From the ROC curve graph, we see a stark difference as well. Both our trained models perform exceptionally well. The kNN model has an AUC of 1.00, meaning it's a perfect classifier for this dataset. The Logistic Regression model is close behind with an AUC of 0.96, showing that it also performs exceptionally well in differentiating between the classes.

When comparing these results to the baseline models (Most Frequent Class and Random Guesses), the baselines are again performing at the expected level of AUC = 0.5, indicating they offer no useful predictive power beyond random guessing.

Hence, for Dataset 2, the best models by far are our trained models, particularly kNN with $k=6$, which has perfect classification capabilities. In this case, the baseline models should be avoided as they offer significantly lower accuracy and predictive power. The noticeable improvement in AUC values and the clear performance difference in the confusion matrices make it evident that using an advanced model like kNN or Logistic Regression is definitely worth it for this dataset.

Index

1.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import KFold, train_test_split
from sklearn.metrics import accuracy_score
from sklearn.neighbors import KNeighborsClassifier
import warnings
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.dummy import DummyClassifier
from sklearn.metrics import roc_curve, auc
```

2.

```
dataset = pd.read_csv('dataset_1.csv', header=None, names=['X_1', 'X_2',
'Target'])
X = dataset[['X_1', 'X_2']].values
y = dataset['Target'].values

kf = KFold(n_splits=5, shuffle=True, random_state=42)

degrees = [1, 2, 3, 4, 5]
C_values = np.logspace(-3, 3, 7)

results = {'degree': [], 'C': [], 'mean_accuracy': [], 'std_accuracy': []}

for degree in degrees:
    poly = PolynomialFeatures(degree=degree)
    X_poly = poly.fit_transform(X)

    for C in C_values:
        accuracy_scores = []
```

```

for train_index, test_index in kf.split(X_poly):
    X_train_cv, X_test_cv = X_poly[train_index], X_poly[test_index]
    y_train_cv, y_test_cv = y[train_index], y[test_index]

    model = LogisticRegression(penalty='l2', C=C, max_iter=10000)
    model.fit(X_train_cv, y_train_cv)

    y_pred_cv = model.predict(X_test_cv)
    accuracy_scores.append(accuracy_score(y_test_cv, y_pred_cv))

results['degree'].append(degree)
results['C'].append(C)
results['mean_accuracy'].append(np.mean(accuracy_scores))
results['std_accuracy'].append(np.std(accuracy_scores))

```

```

results_df = pd.DataFrame(results)

```

```

plt.figure(figsize=(10, 6))

```

```

for degree in degrees:

```

```

    subset = results_df[results_df['degree'] == degree]

```

```

    plt.errorbar(subset['C'], subset['mean_accuracy'],
yerr=subset['std_accuracy'], label=f'Degree {degree}', marker='o')

```

```

plt.xscale('log')

```

```

plt.xlabel('C')

```

```

plt.ylabel('Mean Accuracy')

```

```

plt.title('Cross-Validation')

```

```

plt.legend()

```

```

plt.grid(True)

```

```

plt.show()

```

```

best_index = np.argmax(results_df['mean_accuracy'])

```

```

best_degree = results_df.loc[best_index, 'degree']

```

```

best_C = results_df.loc[best_index, 'C']

```

```

print(f'Best Polynomial Degree: {best_degree}')

```

```
print(f'Best C: {best_C}')
```

3.

```
k_values = range(1, 21)
```

```
results_knn = {'k': [], 'mean_accuracy': [], 'std_accuracy': []}
```

```
warnings.filterwarnings("ignore", category=FutureWarning)
```

```
for k in k_values:
```

```
    accuracy_scores = []
```

```
    for train_index, test_index in kf.split(X):
```

```
        X_train_cv, X_test_cv = X[train_index], X[test_index]
```

```
        y_train_cv, y_test_cv = y[train_index], y[test_index]
```

```
        knn = KNeighborsClassifier(n_neighbors=k)
```

```
        knn.fit(X_train_cv, y_train_cv)
```

```
        y_pred_cv = knn.predict(X_test_cv)
```

```
        accuracy_scores.append(accuracy_score(y_test_cv, y_pred_cv))
```

```
results_knn['k'].append(k)
```

```
results_knn['mean_accuracy'].append(np.mean(accuracy_scores))
```

```
results_knn['std_accuracy'].append(np.std(accuracy_scores))
```

```
results_knn_df = pd.DataFrame(results_knn)
```

```
plt.figure(figsize=(10, 6))
```

```
plt.errorbar(results_knn_df['k'], results_knn_df['mean_accuracy'],  
yerr=results_knn_df['std_accuracy'], marker='o', capsize=5)
```

```
plt.xlabel('Number of Neighbors (k)')
```

```
plt.ylabel('Mean Accuracy')
```

```
plt.title('Cross-Validation: kNN Classifier')
```

```
plt.grid(True)
```

```
plt.show()
```

```
best_k_index = np.argmax(results_knn_df['mean_accuracy'])
```

```
best_k = results_knn_df.loc[best_k_index, 'k']
```

```
best_accuracy_knn = results_knn_df.loc[best_k_index, 'mean_accuracy']
```

```
print(f'Best k: {best_k}')
print(f'Highest Cross-validation Accuracy for kNN: {best_accuracy_knn:.4f}')
final_knn = KNeighborsClassifier(n_neighbors=best_k)
final_knn.fit(X, y)
```

4.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Training the kNN model with k=17
final_knn = KNeighborsClassifier(n_neighbors=17)
final_knn.fit(X_train, y_train)

test_accuracy_knn = final_knn.score(X_test, y_test)
print(f"Test Accuracy for kNN (k=17): {test_accuracy_knn:.4f}")
```

5.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

best_degree = 1
best_C = 0.001
poly = PolynomialFeatures(degree=best_degree)
X_poly_train = poly.fit_transform(X_train)
X_poly_test = poly.transform(X_test)

log_reg = LogisticRegression(penalty='l2', C=best_C, max_iter=10000)
log_reg.fit(X_poly_train, y_train)
y_log_reg_pred = log_reg.predict(X_poly_test)

cm_log_reg = confusion_matrix(y_test, y_log_reg_pred)
disp_log_reg = ConfusionMatrixDisplay(confusion_matrix=cm_log_reg)
disp_log_reg.plot()
plt.title('Confusion Matrix for Logistic Regression')
```

```
plt.show()
```

```
k = 17
```

```
knn = KNeighborsClassifier(n_neighbors=k)
```

```
knn.fit(X_train, y_train)
```

```
y_knn_pred = knn.predict(X_test)
```

```
cm_knn = confusion_matrix(y_test, y_knn_pred)
```

```
disp_knn = ConfusionMatrixDisplay(confusion_matrix=cm_knn)
```

```
disp_knn.plot()
```

```
plt.title(f'Confusion Matrix for kNN (k={k})')
```

```
plt.show()
```

6.

```
dummy_most_frequent = DummyClassifier(strategy="most_frequent")
```

```
dummy_most_frequent.fit(X_train, y_train)
```

```
y_dummy_mf_pred = dummy_most_frequent.predict(X_test)
```

```
cm_dummy_mf = confusion_matrix(y_test, y_dummy_mf_pred)
```

```
disp_dummy_mf = ConfusionMatrixDisplay(confusion_matrix=cm_dummy_mf)
```

```
disp_dummy_mf.plot()
```

```
plt.title('Confusion Matrix for Most Frequent Class Baseline')
```

```
plt.show()
```

```
dummy_random = DummyClassifier(strategy="uniform")
```

```
dummy_random.fit(X_train, y_train)
```

```
y_dummy_random_pred = dummy_random.predict(X_test)
```

```
cm_dummy_random = confusion_matrix(y_test, y_dummy_random_pred)
```

```
disp_dummy_random = ConfusionMatrixDisplay(confusion_matrix=cm_dummy_random)
```

```
disp_dummy_random.plot()
```

```
plt.title('Confusion Matrix for Random Class Baseline')
```

```
plt.show()
```

7.

```

best_degree = 1
best_C = 0.001
poly = PolynomialFeatures(degree=best_degree)
X_poly_train = poly.fit_transform(X_train)
X_poly_test = poly.transform(X_test)

log_reg = LogisticRegression(penalty='l2', C=best_C, max_iter=10000)
log_reg.fit(X_poly_train, y_train)

y_log_reg_probs = log_reg.predict_proba(X_poly_test)[:, 1]

k = 17
knn = KNeighborsClassifier(n_neighbors=k)
knn.fit(X_train, y_train)

y_knn_probs = knn.predict_proba(X_test)[:, 1]

dummy_most_frequent = DummyClassifier(strategy="most_frequent")
dummy_most_frequent.fit(X_train, y_train)
y_dummy_mf_probs = dummy_most_frequent.predict_proba(X_test)[:, 1]

dummy_random = DummyClassifier(strategy="uniform")
dummy_random.fit(X_train, y_train)
y_dummy_random_probs = dummy_random.predict_proba(X_test)[:, 1]

plt.figure(figsize=(10, 8))

fpr_log_reg, tpr_log_reg, _ = roc_curve(y_test, y_log_reg_probs)
roc_auc_log_reg = auc(fpr_log_reg, tpr_log_reg)
plt.plot(fpr_log_reg, tpr_log_reg, label=f'Logistic Regression (AUC = {roc_auc_log_reg:.2f})')

fpr_knn, tpr_knn, _ = roc_curve(y_test, y_knn_probs)
roc_auc_knn = auc(fpr_knn, tpr_knn)
plt.plot(fpr_knn, tpr_knn, label=f'kNN (k={k}, AUC = {roc_auc_knn:.2f})')

```

```
fpr_dummy_mf, tpr_dummy_mf, _ = roc_curve(y_test, y_dummy_mf_probs)
plt.scatter(fpr_dummy_mf[1], tpr_dummy_mf[1], color='red', label='Most Frequent
Classifier')

fpr_dummy_random, tpr_dummy_random, _ = roc_curve(y_test, y_dummy_random_probs)
plt.scatter(fpr_dummy_random[1], tpr_dummy_random[1], color='green',
label='Random Classifier')

plt.plot([0, 1], [0, 1], 'k--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve Comparison')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```